FORGE / ENGINEERING READINESS

Production readiness

Expanded review and implementation guide

28 October 2026 | Revision ededa613

Not ready for collaborative production.

The feature gaps are correctly identified. This revision adds the implementation contracts, recovery behavior and release evidence needed to close them.

What changed

14 added or expanded risk areas. A reconstructed 31-area inventory. Domain and migration guidance. Transaction, agent, storage, job, API and MCP contracts. Release gates and an evidence register.

Reading guide

01-04 Assessment, corrections, inventory and missing risks

05-12 Implementation guidance by engineering area

13-15 Operations, delivery sequence and source evidence

Scope: source-backed review and read-only deployment inspection. No application changes or production certification. The original PDF remains unchanged.

01. Decision and audit boundary

Forge is not ready for a collaborative production launch. The original report correctly identifies the main blocker: core work objects remain browser-local. Its feature inventory is useful, but it does not yet define the consistency, recovery, authorization, migration, and operational guarantees needed to turn the inventory into a delivery plan.

This revision reviews the supplied seven-page PDF against the clean local checkout at `ededa613588aaf885732ec05f6ecbf9cb17f43a4`, the same revision named in the original. It adds implementation guidance; it does not implement those changes or certify the deployed application.

Evidence limits. The PDF's feature table is horizontally clipped on pages 2-3, and the document ends during Open questions without the advertised Sources section. Missing cells cannot be recovered from the PDF. The matrix below reconstructs the visible inventory and distinguishes directly checked code from original-report claims. The linked Grok page required sign-in and was not readable.

Verification performed. Source inspection of persistence, auth, API idempotency, agent admission/streaming/storage, Calls retention, skills, migrations, and CI. Read-only Vercel project and latest-deployment lookups. Twenty existing tests passed across three files: agent control policy, API idempotency, and agent coordinator. No production database query, migration, live model invocation, provider OAuth exercise, load test, full build, or full test suite was run.

Recommended release scope (proposal). Start with an invited collaborative pilot: accounts, workspace membership, teams, issues, comments, basic projects, durable files, and server inbox. Include Calls, agent execution, public API, or desktop only when their individual gates pass. MCP, paid billing, advanced planning, and analytics integrations need not block that pilot if their unavailable surfaces are explicitly disabled. A commercial release has additional billing and support requirements.

Priority definitions. P0 blocks the release scope that depends on it. P1 is required before broad rollout or before enabling the affected optional feature. P2 is subsequent capability work. These are proposed priorities, not promised dates or staff assignments. The original's 8/13/10 counts are not reused: features and launch risks are different units.

Core change to the plan. Deliver a complete vertical slice before expanding every domain: authorized issue write -> transaction -> durable event -> second user's view -> API -> approved agent write. Establish storage, recovery, monitoring, and migration controls alongside that slice, not after feature completion.

02. Corrections to the original report

Deployment provenance needs correction. The live Vercel lookup returned latest production deployment `dpl_DHbhSDCAubnVvndVKDUgvCCzH5Bu`, READY, source `git`, region `iad1`. Therefore “CLI uploads straight to production” is not an accurate description of this latest deployment. This response does not establish preview deployments, passing CI gates, branch protection, or the deployed commit; metadata did not contain a commit SHA. Verify those separately. Project protection remains `all_except_custom_domains`; the returned domains are Vercel domains. Configure the intended audience and domain deliberately rather than treating a custom domain as the only possible remedy. [E20]

The empty Neon project is not evidence that the serving database is empty. The production connection target remains unresolved here. The README names a different project from the one in the original report. Map deployment -> secret reference -> hostname -> project/branch/database, then inspect only schema and migration metadata. Do not print credentials or run migrations against whichever project happens to be visible. [E01]

Missing environment variables are feature-dependent. The API can remain disabled for a pilot. Desktop release variables belong to the desktop release pipeline; Calls OAuth belongs to enabled providers. `drizzle.config.ts` prefers `DATABASE_URL_UNPOOLED` and explicitly falls back to `DATABASE_URL`; its absence alone does not prove runtime failure. Build a per-service/per-environment contract with required, conditional, and optional settings. Forge currently uses its own Better Auth; provisioning managed Neon Auth does not migrate Forge sessions. [E01, E17]

Existing controls are not universal guarantees. Application services and tests are real, but not every domain has identical audit, idempotency, or rate-limit behavior. The API and capability wrappers have separate reservation, mutation, and result writes. Passing fixture evals is not evidence of live provider or deployed sandbox health. [E06, E07, E18]

“No cron” is too broad. GitLab defines scheduled product retention and workflow-health jobs, conditional on schedule variables. Calls sync and agent retention are not wired into those jobs. Configuration proves an intended mechanism, not that a schedule is enabled or succeeding. [E16]

Persisting skills and flipping a flag is insufficient. The skill flag exposes a composer menu of built-in capabilities. Server loading of the saved personal/team instructions and their versioned inclusion in a run still need implementation and tests. [E14]

Secrets and devtools findings need evidence. A Notion page title does not prove it contains live credentials. Inspect access and rotate only if exposure is established. An unconditional devtools component in source does not prove production code survives bundling; inspect a fresh production artifact. Neither was verified here.

03. Reconstructed feature inventory

Legend: **checked** = relevant owning source inspected in this review; **baseline** = carried from the supplied PDF, not independently re-audited in depth. “Server” means a server implementation exists, not that production has passed acceptance.

#	Area	Current boundary and required next step
1	Authentication	Server; checked. Add recovery and define verification policy; test deployed flows.
2	Workspace, members, invites, roles	Server; baseline with actor/CI checks. Prove revocation and invitation delivery.
3	Teams and settings	Server; checked service/API wiring. Extend access tests to dependent objects.
4	Issues and detail	Browser-local; checked. Canonical issue, comment, reaction, file, history and subscriber services.
5	My issues	Local issue-derived view; checked dependency. Query the same authorized issue service.
6	Triage	Browser-local; baseline. Model triage as issue lifecycle, avoid a second issue store.
7	Statuses/labels/workflows	Browser-local; checked issue reads. Server-owned definitions and transition rules.
8	Projects/milestones/updates	Browser-local; checked project store. Persist relationships and project activity.
9	Cycles/upcoming	Browser-local; checked. Shared issues, explicit membership, timezone-aware rollover.
10	Initiatives	Browser-local; baseline. Persist project membership and rollups.
11	Views/favorites	Browser-local; checked. Versioned filter definitions; private/shared visibility.
12	Documents/resources/loops	Local documents confirmed; loops baseline. Persist rich text and revisions.
13	Inbox/notifications	Browser-local; checked. Durable events, recipients, subscriptions, read/snooze state.
14	Calls	Server; checked. Prove OAuth/scheduling and issue conversion; define derived-data retention.
15	Templates	Server; schema checked. Instantiate through canonical commands with current permissions.
16	Recurring issues	Server schedule configuration only; checked. Idempotent occurrence generation is missing.
17	Custom emojis	Server; schema checked. Verify upload limits and rendered-content policy.
18	Agent conversations	Server; checked. Fix admission/recovery guarantees before enabling production writes.
19	Agent artifacts	Metadata in DB, bytes on local disk; checked. Durable private storage required.
20	Model connections	Server encrypted credentials; checked. Rotation, provider smoke tests and spend controls.
21	Personalization/team skills	Local; checked. Server scope, revisions, run-context assembly and injection tests.
22	Public API/keys	Teams API exists; checked. Expand scopes/contracts and harden idempotency.

#	Area	Current boundary and required next step
23	Workspace security	Server; checked entry points. Exercise policy parity across every transport.
24	Signals	Explicit fictional preview; checked. Real provider ingestion is future work.
25	Reviews	UI without live PR data; baseline. Gate until provider identity/access/sync exists.
26	Insights	Placeholder; baseline. Gate until a real data and analysis contract exists.
27	Billing/plans	UI only; baseline. Required for paid launch, optional for a capped free pilot.
28	Integrations/accounts/apps	UI-only group in PDF; baseline. Do not confuse with implemented Calls OAuth.
29	Import/export	UI only; baseline. Separate local-data migration from user-facing export jobs.
30	Preferences, profile extras, SLA	Mostly local per PDF; baseline. Keep cosmetic device preferences local; persist shared policy.
31	macOS and updates	Implementation/release pipeline exists; checked docs/CI. Signed packaged release still needs evidence.

04. What the report missed or under-specified

ID	Priority	Addition and consequence
R1	P0 core	Tenant-safe relationships and authorization on search, files, events, and imports; a workspace filter on one route is insufficient.
R2	P0 core	Concurrent edits, issue numbering, deletion semantics, and a safe browser-data migration; a CRUD backend alone can lose or duplicate work.
R3	P0 writes	Crash consistency across business mutation, idempotency result, audit and notification event. Existing wrappers do not commit these together.
R4	P0 agents	Atomic quota admission and reserved spend. Current read-then-create flow can exceed limits under simultaneous starts.
R5	P0 files	Hosting request limits and direct uploads. A permitted 5 MiB attachment encoded as JSON exceeds the documented Vercel request ceiling.
R6	P0 agents	Explicit execution ownership, request lifetime, cancellation, crash recovery and paged event replay. Durable rows alone do not prove durable execution.
R7	P0 core	Backup/restore rehearsal, schema rollback compatibility and environment isolation. “Run migrations” lacks a recovery plan.
R8	P0 enabled jobs	Durable job identity, retries, leases, missed-run alerts and occurrence deduplication; merely adding a cron is insufficient.
R9	P1 broad launch	Account recovery, shared abuse controls and immediate access revocation across sessions, keys, files and agents.
R10	P1 Calls	Retention of summaries, notes, actions, exports and backups as well as transcripts; deletion promises need a full data lifecycle.
R11	P1 integrations	API/MCP authorization and approval policy are distinct. API-key auth does not automatically satisfy interoperable MCP OAuth.
R12	P1 rollout	Release evidence: deployed multi-user tests, live provider tests, measurable service objectives, accessibility and load budgets.
R13	P1 credentials	Key rotation and recoverability. The vault's payload format has no key identifier for multiple active encryption keys.
R14	P1 skills	Persisted instructions need trust boundaries, versioning, explicit run snapshots, size limits and rollout controls.

R3-R6 and R13 have concrete code evidence below. Other additions are required design/validation work, not assertions that an untested production exploit has occurred. Several topics were mentioned briefly in the PDF; the improvement is specifying how they work and how to prove them.

05. Core domain implementation: one source of truth

Entry points. Add application services beside `application/teams`, schema beside `src/db/schema.ts`, and adapters beside the existing auth/API adapters. Use the established Actor and typed error patterns. UI server functions, public API, and agent tools must call the same domain commands; none should implement an independent issue mutation. [E02-E04]

Minimum schema (proposed). Create issues with immutable UUID, workspace/team IDs, team-local sequence number, title/body, state, priority, assignee, optional project/cycle/parent, version, creator, timestamps and deletion marker. Normalize status definitions, labels and issue-label links. Add comments, reactions, subscriptions, attachments and immutable activity. Projects/cycles must exist as minimal referenced entities before a slice enables those fields; defer advanced planning attributes.

Integrity rules. Allocate `(team_id, number)` inside the creation transaction with a unique constraint. Scope foreign keys or equivalent validated relations so issue, project, cycle, parent, status, label and attachment cannot cross workspaces. Reject incompatible team/project/cycle assignments and parent cycles. Decide whether projects can span teams before encoding constraints. Do not map a team rename to a new issue identity.

Authorization. Derive workspace and user from the authenticated actor, never request JSON. Resolve current membership, private-team visibility, role, scopes, resource ownership and policy for every read and write. Filter before pagination and aggregation, not after. Apply the same rule to exports, search snippets, attachment downloads, event subscriptions, bulk actions and agent tools. Recheck permissions when deferred work actually executes.

Concurrent edits. Require `expectedVersion` on updates. Use an atomic update conditioned on current version and return a typed conflict with the newest authorized record. Apply optimistic UI patches only as pending changes; restore or reconcile on failure. Define bulk-operation atomicity and partial-failure results. Rich-text documents need revisions and conflict handling; real-time co-editing can be a separately scoped CRDT project.

Transaction boundary. Commit business rows, history, command receipt and outbox events together. Cache invalidation happens only after commit. Worker delivery reads committed outbox entries and is retryable. UI success means the server accepted the write, not merely that a local optimistic row appeared.

Acceptance gate. Two signed-in users on independent browser profiles see the same issue; two simultaneous edits cannot silently overwrite; cross-workspace/private-team references fail; duplicate create retries return one issue and one number; delete/restore semantics propagate to search and subscriptions. Add integration tests against an isolated real Postgres database, not only mocked adapters.

06. Migrate browser data without guessing ownership

The original's “one-time import” needs its own protocol. Several stores are keyed by team, while others use workspace/user scope. A record in this browser is not proof that the current account created it. Inventory keys and schemas first, including issue statuses, labels, cycles, projects, documents and issue attachments. [E02]

1. **Discover and preserve.** Read local data without changing it. Offer a local backup and a preview showing counts, destination workspace, unresolved references and excluded records. Do not auto-upload every origin key at login. Quarantine malformed JSON instead of replacing it with an empty collection.

2. **Validate ownership.** Require a signed-in member with destination import permission. The user explicitly selects the destination for ambiguous legacy records. Revalidate each referenced team and user on the server. Treat imported actor names, dates and HTML as untrusted historical data.

3. **Create an import manifest.** Store a schema version, import ID, source fingerprint, destination, per-entity legacy IDs, checksums, states and errors. A uniqueness constraint such as `(workspace_id, import_id, entity_kind, legacy_id)` makes retries safe. A repeated file with a different import ID also needs duplicate detection or an explicit review choice.

4. **Import dependencies in order.** Resolve definitions and planning objects; map issues and parents; import comments and relationships; upload files through the normal validated storage path. Allocate server issue numbers and preserve old IDs as migration aliases. Do not trust client-supplied workspace IDs or overwrite existing server objects by matching only a title.

5. **Handle concurrent local changes.** Record the source snapshot hash. If it changes, import a reviewed delta or require another preview. Do not maintain indefinite local/server dual-write authority. Per-workspace cutover flags should route all enabled issue entry points to the server together.

6. **Reconcile and switch.** Compare counts, relationships and file checksums. Show rejected records and allow resumable retries. Keep local backup data until reconciliation and explicit cleanup; restarting the browser must not duplicate imported objects.

Acceptance gate. Test interrupted uploads, partial batches, two browser profiles importing overlapping data, changed legacy IDs, quota failures, malformed JSON, revoked membership mid-import and rollback to the read-only backup. A migrated call action or Signals draft must link to a real server issue ID.

Preference policy. Keep layout widths, temporary drafts and device-specific display settings local. Persist notification subscriptions, team workflows, shared views, SLA rules and skill policy. Cache keys for server data must include actor/workspace scope and be cleared on account/workspace changes; stale caches must not reveal a previous user's content.

07. Fix idempotency at the transaction boundary

Observed gap. `executeIdempotent` reserves a key, calls `run()`, then updates the stored response. The business mutation is outside the receipt update transaction. A crash after the mutation can leave “in progress” until expiry; after expiry, the same logical request can run again. Its error path deletes the reservation even though the wrapper cannot prove every possible operation rolled back. [E06]

The agent gateway similarly reserves an invocation, consumes approval, executes the capability, and completes the reservation in separate operations. A failure between those steps can strand an approval or leave an ambiguous result. Existing deduplication and input hashes are useful; they do not eliminate the crash windows. These are source-based failure analyses, not a reproduced production incident. [E07]

Implementation. Introduce an application command transaction that accepts Actor, operation ID, canonical input hash and command ID. Acquire/insert the receipt; validate current authorization and expected version; apply the business mutation; insert audit/activity and outbox rows; persist the result; commit. Expose a transaction-aware repository interface so the wrapper and domain operation use the same transaction. Keep database transactions short and never hold them while calling model or mail providers.

Agent approval. Bind approval to actor, workspace, operation/version, exact input hash and expiry. For a database-only write, approval consumption and mutation should share the transaction where feasible. Otherwise use explicit claimed/executing/completed states with a recoverable command identity; never silently release an approval after an uncertain effect. Recheck current permissions even for a previously approved operation.

External effects. Use an outbox and provider idempotency keys where supported. Record the provider's stable reference and reconcile ambiguous responses. Do not promise exactly-once email or webhook delivery; provide at-least-once delivery and consumer deduplication. Apply a retention window long enough for documented retries and maintain durable business uniqueness beyond a short receipt TTL.

Recovery. An expired lease means “needs reconciliation,” not “safe to run again.” Return a retryable pending result while checking a durable command ID. Store typed failures without secrets. Apply bounded retries and alert on stale receipts; operators need a safe reconciliation command and audit trail.

Acceptance gate. Kill the worker after reservation, after approval consumption, after the business commit, before receipt completion and after a provider accepted the request. Retrying the same command must produce one durable business effect and an inspectable outcome. Include two concurrent identical requests, conflicting payload reuse, revoked membership, and retention-boundary retries. Existing unit tests passed, but these fault-injection gates were not run.

08. Agent admission, lifetime and spend

Observed quota race. `startTurn` calls usage/provider checks before inserting the run. These are separate database operations, so simultaneous starts can both see available capacity. `summarizeUsage` also applies a month-start filter to the same query that counts active runs; an active run begun just before the month boundary is omitted from the next month's user/workspace active count. Provider counting is separate. Budgets check recorded spend, without an admission-time reservation for future cost. [E08]

Implementation. Make admission a single transaction: lock per-workspace/user/provider quota records in a consistent order, validate active leases and available budget, reserve bounded spend, and create the run. Count concurrency independently from monthly accounting. Release reservations idempotently on terminal state; reconcile unknown provider usage and expired worker leases. Define whether “provider limit” means per workspace or shared across all customers. Unknown custom-model pricing must not silently count as free.

Observed execution risk. The HTTP handler consumes the event iterator inside a streaming response; the configured agent wall-clock default is 15 minutes. Vercel limits cover streamed-response time too. Current documentation lists plan/configuration-dependent limits, including an extended-duration beta, so do not assume either a universal timeout or guaranteed 15-minute support. Read the actual deployed function settings. [E09, W1]

Execution design. Separate run execution ownership from the browser's stream. Use a durable worker/workflow or a proven runtime-owned job, heartbeat lease, cancellation flag and persisted cursor. Reconnect should read persisted events and attach to the same execution; it must not start a second writer. Keep the existing EVE abstraction. Explicitly define disconnect behavior, deadline propagation, approval waits and worker takeover. Verify the deployed sandbox backend and deny-all egress with a live, bounded test.

Event scalability. The Drizzle event store retries sequence allocation from `max(sequence)` and reads replay/history without a page limit. Add bounded pages, indexed cursor access and a stable cursor contract. Thread-wide replay cannot treat a per-run sequence as a globally unique thread position. Benchmark contention before changing sequence allocation; retain source-event deduplication. [E09]

Acceptance gate. At a cap of one, simultaneous starts across two processes admit one run. Cross a month boundary with an active run. Disconnect/reconnect and redeploy during execution; no duplicate tool effect. Cancel during approval and provider streaming; reservations eventually settle. Show limits, failure and recovery to the user. Instrument active leases, recovery age, reserved/settled cost and approval latency without logging prompts or credentials.

09. Durable files, upload limits and deletion

More than artifacts are affected. `forge-capabilities.server.ts` injects the same `LocalArtifactBlobStore` into both artifact and agent-attachment services. Metadata is durable but bytes remain under `.forge/agent-artifacts` by default. Replacing artifact downloads alone leaves uploads and model attachment resolution dependent on disk. [E10]

Concrete size mismatch. The default attachment limit is 5 MiB; the input carries base64 JSON. That becomes roughly 7 MB before JSON overhead, exceeding Vercel's documented 4.5 MB function payload limit. The browser can submit a file accepted by Forge's limit but rejected before the service executes. Merely changing the blob adapter does not fix that ingress path. [E10, W1]

Proposed flow. Create an authorized upload intent containing workspace, actor, purpose, size, MIME constraints, checksum and expiry. Reserve quota. Issue a short-lived private-object upload URL; upload bytes directly; finalize through Forge after independently checking the stored object metadata and content policy. Mark objects pending/ready/rejected/deleting so an incomplete upload is never a usable attachment. Only ready objects can be linked to a permitted issue or run.

Access. Use non-guessable object keys and private buckets. Recheck membership/resource access when issuing a short-lived download grant. Define the revocation window for already issued URLs; use an authorized proxy when immediate revocation is mandatory. Separate untrusted HTML/SVG previews from the application origin, use safe response headers, and bound archive expansion and document extraction. Validate content rather than trusting filename or client MIME alone.

Lifecycle. Share the storage interface across agent attachments and artifacts, with separate retention policies from issue/document files. Preserve checksum metadata. Reconcile orphaned objects and metadata. Deletion jobs need tombstones, retries and durable progress; deleting a row must not silently orphan bytes. Backups and restore procedures must cover object storage as well as Postgres.

Acceptance gate. Upload a file near each configured boundary, then read it from another application instance and after redeploy. Test interrupted finalize, repeated finalize, bad MIME/checksum, private-team removal, expired grants, full quota, deleted parent and retention failures. For server-to-model delivery, keep bounded content resolution or provider-compatible file delivery; direct browser upload alone does not remove downstream model-context or request limits.

10. Jobs, notifications, Calls and recurring work

Job substrate. Reuse existing Workflow SDK patterns where appropriate; choose one owner for each scheduled task. Record job key, scheduled time, attempt, lease owner/expiry, checkpoint, error and completion. Every worker must tolerate duplicate invocation and resume bounded batches. Use backoff with jitter, retry ceilings and dead-letter/reconciliation procedures. Scheduler endpoints require authentication; feature flags must guard server execution as well as UI. [E16]

Notifications. In the issue transaction, append an outbox event with workspace/resource ID, actor, event version and sequence. Resolve recipients with current access and subscription policy. Keep inbox delivery and outbound email as separate idempotent effects. Make read, snooze and deletion state server-backed; replay with a cursor after reconnect. Do not put private issue bodies in email before checking recipient access. Add delivery failures, bounces and suppression handling. The existing `publishInboxUpdate` adapter is a UI integration point, not authorization. [E11]

Recurring issues and cycles. Persist schedule timezone, next occurrence and explicit daylight-saving behavior. Create each occurrence with a unique `(schedule_id, occurrence_time)` command identity. Define catch-up after downtime, edited/disabled schedules, deleted teams and template version selection. Cycle rollover must use canonical issues and preserve unfinished-work history. Configuration tables already exist; instance generation does not. [E03]

Calls. Preserve existing OAuth state binding, token encryption and sync/summary leases. Add scheduler evidence, token-refresh race handling, reconnect states, bounded backfills and alerting on stale sync. When users convert actions to issues, use the issue command service and a durable call-action/issue relationship, not a browser-only backlink. Deduplicate that conversion under retries. [E12]

Retention detail missing from the report. The current transcript sweeper clears transcript text and marks calls expired. It leaves summary and decisions on the call and does not delete notes/actions in that operation. This can be deliberate, but “retention” must explain it. Define independent lifetimes for raw transcript, derived summary, notes/actions, exports, model events, audit logs and backups. Define account/workspace deletion and re-import behavior. This is a product/data-lifecycle requirement, not a legal compliance certification. [E12]

Acceptance gate. Execute duplicate/overlapping jobs, crash midway, revoke a connection, expire a lease, cross daylight-saving boundaries, and simulate a week of downtime. Verify one issue per occurrence, no duplicate notification, recoverable imports, and a visible alert when no successful sweep has occurred within the agreed interval.

11. Public API, Forge tools and MCP

API first means shared services first. Extend the oRPC contracts and application commands for issue search/get/create/update/comment, projects and cycles. Add explicit scope names and error contracts, pagination with stable tie-breakers, bounded filters, response-size limits, version preconditions and idempotency semantics. Preserve the disabled-by-default production API until authorization and deployment tests pass; enabling the flag is a release action, not a fix by itself. [E04]

Webhooks. Commit event production with the originating write; deliver asynchronously. Define event IDs/schema versions, payload minimization, signature timestamps, secret rotation, replay windows and redelivery. Validate destination URLs, prevent private-address/redirect abuse, bound time and response sizes, and recheck workspace access at registration. Test signature verification and duplicate/out-of-order delivery.

Agent tools. Add typed capabilities that call the same services. Read tools filter by actor access. Mutations use exact-input approvals and the crash-safe command model from section 07. Treat issues, documents, meeting transcripts and tool results as untrusted input; do not let retrieved text modify policy or destination permissions. Update instructions and fixture evals, then run live, capped tests through the deployed model/runtime/sandbox. A fixture eval cannot validate real credentials or provider behavior. [E07, E18]

MCP server is a separate product surface. Implementing external MCP access is optional for a collaborative pilot. For a hosted authenticated server, implement the chosen MCP protocol version and compatible transport, discovery, OAuth resource authorization, resource/audience validation and per-request access checks. The current MCP authorization specification requires protected-resource metadata when that authorization model is used and rejects tokens intended for another resource. A manually configured API-key mode may serve selected clients; it is not proof of general OAuth client compatibility. [W2]

Approval policy must be explicit. User-initiated browser writes, scoped API automation and autonomous agent writes have different interaction models. An external MCP client may not have a Forge run ID or the internal approval UI. Start with read-only MCP. For writes, either define explicit narrowly scoped delegated authorization or create a Forge pending-action/approval exchange and execution identity. Never map an external token to an internal agent run or treat an MCP tool annotation as authorization.

Acceptance gate. Contract tests prove the same access, mutation and error behavior through UI, API and agent paths. MCP tests cover expired/wrong-audience tokens, missing scopes, revoked membership, reconnects, output-size bounds and a representative supported client. Outbound MCP connectors need a separate credential, SSRF, tool-discovery and approval design; keep the existing toggle hidden until implemented.

12. Authentication, secrets and trust boundaries

Recovery and verification. `auth.ts` enables passwords and configures verification mail, but does not configure a password-reset mail handler or require verified email for password sign-in. Define the pilot's verification policy and implement the complete recovery flow: request, generic response, single-use expiry, password change, session policy and success email. Exercise email change, account linking, expired invitations and member removal. The current configuration alone does not establish a vulnerability; it establishes unfinished recovery/product policy. [E13]

Authorization parity. Preserve database-backed session validation and current Actor construction. Test removed or downgraded members with an old tab, API key, approval, file URL and active agent run. Define cancellation on lost access. A team/workspace mismatch must be denied across direct IDs, search, metrics, bulk mutations and asynchronous jobs. Add CSRF/origin tests for cookie-authenticated writes and explicit allowed-origin/CORS rules for API transports.

Shared abuse controls. Keep API-key throttling and add or verify distributed limits for sign-in, verification/reset mail, invitations, uploads, expensive search, model discovery and agent starts. Check the installed Better Auth limiter behavior/configuration before claiming it is memory-only. Define behavior when the limiter store is unavailable. Logs must avoid raw tokens, private bodies and full signed URLs.

Credential rotation. The vault encrypts with a single configured 32-byte key and emits a version/nonce/tag/ciphertext record without a key ID. Simply replacing the environment key makes old values unreadable. Add a key identifier and a versioned keyring; write with the current key, read with allowed older keys, re-encrypt in resumable batches, verify coverage and only then retire old keys. Include Calls tokens and BYOK credentials. Back up encryption keys securely with the recovery plan. [E15]

Skills and personalization. Persist personal instructions by workspace/user and team skills by authorized team/revision. Load only explicitly applicable skills, bound their size and snapshot the revisions used in each run. Treat skill content as lower-trust data beneath system and authorization policy; it cannot grant new tool permissions. Review who may publish a team skill. Audit selection and revision metadata without copying sensitive instructions into general diagnostics. The feature flag is not this integration. [E14]

Acceptance gate. Recover a test account; reject a reused reset token; revoke a member during a pending write; rotate encryption keys without losing old credentials; reject a malicious skill that requests another workspace's data. These tests belong in isolated environments with test credentials, not the production customer dataset.

13. Deployment, observability and recovery

Establish identity before deployment. Record application and runtime project IDs, production aliases, deployment IDs, commit SHAs, region, database host/project/branch/database, schema version and storage bucket. Keep values such as keys and URLs containing passwords out of the evidence artifact. The current Vercel deployment lookup proves READY and Git source only; it does not prove the serving database or function settings. [E20]

Migration/release flow. Use a staging database and test credentials isolated from production. Rehearse migrations on a representative sanitized copy. Run additive expand migrations before compatible application code, backfill in bounded restartable batches, switch reads/writes behind a scoped flag, then remove old columns in a later release. Serialize migration ownership. Do not use `db:push` as an uncontrolled production release step. Verify an application rollback can still read the new schema.

Recovery. Set proposed pilot objectives of recovery point <= 1 hour and recovery time <= 4 hours; the owner must accept or revise these before launch. Verify the configured database restore window, file backup/version policy and encryption-key recovery. Restore into isolation, reconcile schema/object checksums, and exercise the application against the restored copy. Record actual restore duration and data loss. Backups without a tested restore do not satisfy this gate.

CI and release evidence. Retain lint, typecheck, tests, builds, fixture evals and packaged desktop tests. Add real-database authorization/transaction tests and a deployed multi-user smoke test. Check that the exact deployed commit passed required checks. The presence of GitLab jobs, a GitHub workflow or a Git-sourced Vercel deployment does not prove required checks or protected promotion. Record preview/staging/prod configuration differences. [E16]

Observability from the first slice. Correlate requests, commands, jobs, run IDs and deployment IDs. Measure API errors/latency, DB pool saturation, event backlog age, stale leases, failed imports, mail delivery, upload failures, model spend and scheduler freshness. Keep payloads redacted. Add liveness, dependency readiness and synthetic account/issue flows; readiness must fail when migrations or required dependencies are incompatible.

Proposed pilot service objectives. 99.5% successful core requests over a rolling 7-day window; p95 issue list <= 500 ms and writes <= 1 s at the agreed pilot workload; notification delivery <= 60 s when dependencies are healthy. These are proposed budgets, not measured results. Define dataset size, region, concurrency and dependency exclusions before testing. Include keyboard-only core flows, mobile layouts, focus/error handling, timezone handling and slow/offline behavior.

Desktop and supply chain. If shipping desktop, verify signed/notarized packages and updates for both supported architectures, update-feed integrity, release pause/rollback and IPC/navigation policy. Track dependency scanning, license/SBOM output and patch ownership. Inspect a fresh production bundle for devtools rather than inferring inclusion from source.

14. Delivery sequence and exit gates

Owners below are proposed roles; assign actual people before scheduling. Avoid time estimates until schema and release scope are agreed.

Slice	Owner role	Requires	Required proof before advancing
A. Environment and recovery	Platform	None	Serving DB identified, isolated staging, restore rehearsal, per-service config contract, basic alerts.
B. Transaction/storage foundation	Backend + platform	A	Tenant-safe schema, command receipts/outbox, direct private uploads, failure-injection tests.
C. Collaborative issue slice	Product backend + frontend	B	Two-user issue/comment flow, version conflicts, access revocation, migration preview/reconciliation.
D. Minimal planning/inbox	Product backend + frontend	C	Same issue IDs across projects/cycles/My issues, durable inbox, duplicate-safe notification worker.
E. Optional agent/API rollout	Agent + API owners	B-C	Atomic admission, crash-safe approved writes, paged replay, deployed capped model/sandbox tests.
F. Calls and recurring work	Integrations + jobs owner	B-D	OAuth refresh/reconnect, action-to-issue identity, scheduler freshness, retention and occurrence tests.
G. External MCP	API/integrations	C, E	Read-only client compatibility, OAuth/scopes, explicit policy before enabling writes.
H. Commercial/advanced scope	Product + platform	Relevant prior slices	Paid entitlement reconciliation if charging; real provider data or disabled previews.

Dependency corrections. Durable storage, observability, scheduled-job health and restore controls move earlier than in the original sequence. Recurring generation waits for canonical issue commands. Agent/API expansion waits for shared domain services, but does not require every advanced planning screen. MCP waits for a deliberate external authorization model; it is not the next automatic step after enabling an API flag.

Pilot go/no-go. Release only after A-D pass and every enabled optional feature passes its own gate. Record evidence at a fixed commit/deployment and demonstrate: two users; forbidden cross-workspace/private-team reads; concurrent edit conflict; retry after ambiguous failure; import recovery; file availability after redeploy; notification replay; recoverable account access; successful restore; alert delivery. Disable unready modules server-side and label remaining sample previews accurately.

Expansion. Add advanced projects/milestones, initiatives, documents/revisions, shared views and analytics only after the issue spine is stable. If billing is enabled, verify signed webhook handling, duplicate and out-of-order event reconciliation, subscription cancellation/grace rules and server-side entitlements. A successful checkout redirect must never be the authority that grants a plan.

Remaining decisions. Intended pilot audience and enabled surfaces; authoritative production database; project/team visibility model; import ownership policy; data/derived-data retention; scope of agent spend caps; delegated MCP write policy; actual recovery/service objectives; owners and release date. None requires guessing credentials or publishing the report externally.

15. Evidence and verification register

All code references below are relative to the repository at commit `ededa613588aaf885732ec05f6ecbf9cb17f43a4`. Paths identify the owning source; recommendations elsewhere in this document are proposed changes. The original PDF is the baseline for carried-forward inventory rows, not proof of current infrastructure.

Ref	Owning source / evidence
E01	`README.md` Database section; `apps/forge/drizzle.config.ts` connection selection.
E02	`apps/forge/src/lib/team-issues.ts` read/write persistence; `projects-state.ts`, `team-cycles.ts`, `custom-views-state.ts`; `docs/documents.md`.
E03	`apps/forge/src/db/schema.ts` domain inventory and recurring-issue configuration comment.
E04	`apps/forge/src/api/v1/router.server.ts`; `src/api/v1/handler.server.ts`; `src/adapters/auth/session-actor.server.ts`; `src/application/authorization.ts` (all under apps/forge).
E05	`apps/forge/src/lib/signals-accounts.ts`, `signals-insights.ts`, `signals-documents.ts`; explicit fictional data and local drafts.
E06	`apps/forge/src/api/v1/idempotency.server.ts`, function `executeIdempotent`: reserve -> run -> result update.
E07	`apps/forge/src/application/agents/capability-gateway.server.ts`, method `invokeWrite`; `src/adapters/agents/drizzle-capability-idempotency-store.server.ts` (under apps/forge).
E08	`apps/forge/src/application/agents/agent-coordinator.server.ts`, `startTurn`; `agent-control-policy.ts`; `src/adapters/agents/drizzle-agent-store.server.ts`, `summarizeUsage/createRun`.
E09	`apps/forge/src/application/agents/agent-http-handler.server.ts`, `streamEvents`; `agent-control-policy.ts`; `src/adapters/agents/drizzle-event-store.server.ts`.
E10	`apps/forge/src/adapters/agents/local-artifact-blob-store.server.ts`; `forge-capabilities.server.ts`; `src/application/agents/attachment-service.server.ts`, `attachment-contract.ts`.
E11	`apps/forge/src/lib/inbox-state.ts`; `docs/inbox.md` (some prose is stale; current stores govern this review).
E12	`apps/forge/src/application/calls/calls.server.ts`, especially `sweepCallTranscripts`; `docs/calls.md`.
E13	`apps/forge/src/lib/auth.ts`; `src/adapters/auth/session-actor.server.ts`.
E14	`apps/forge/src/lib/team-agent-skills.ts`, `agent-personalization.ts`; `src/components/agent/agent-composer.tsx`; `src/application/agents/agent-feature-flags.ts`.
E15	`apps/forge/src/application/agents/model-secret-vault.server.ts`, `EnvelopeModelSecretVault`.
E16	`.gitlab-ci.yml`; `.github/workflows/release-macos.yml`; `apps/forge/scripts/sweep-product-retention.ts`; root `package.json`.
E17	`apps/forge/vite.config.ts`; `services/agent-runtime/package.json`; `services/agent-runtime/agent/sandbox/sandbox.ts`. Runtime README pin is stale (0.27.1 versus package 0.31.0).
E18	`services/agent-runtime/scripts/run-evals.mjs` enables fixture mode; `evals/first-milestone.eval.ts`; original completion audit is historical.
E19	Local targeted Vitest result: 3 files / 20 tests passed on 28 October 2026; no new concurrency or fault-injection tests added. Counted 206 test files and 26 SQL migrations, matching the original inventory counts.

Ref	Owning source / evidence
E20	Read-only Vercel get_project/get_deployment on 28 October 2026: project `prj_S0XLtdngbexhl2ao7WviBrlMScar`; deployment `dpl_DHbhSDCAubnVvndVKDUgvCCzH5Bu`; READY, source git, iad1, protection all_except_custom_domains.

W1. Vercel Functions Limits. Consulted 28 October 2026: streamed responses count toward invocation duration; payload limit is documented as 4.5 MB; duration depends on plan and configuration. https://vercel.com/docs/functions/limitations

W2. MCP authorization specification, 2026-07-28. Consulted 28 October 2026 for hosted authorization, metadata and audience validation. Pin and test the chosen protocol version rather than following an unversioned “latest” reference in implementation. https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization

Original supplied source: `forge-app production-readiness report.pdf`, seven pages, supplied from Downloads. Source URL: https://forge-app-production-readiness-report-ew2f.pages.grok.bot/ (sign-in blocked during this review). Its Notion secrets, production environment-variable inventory, Neon contents, runtime deployment age and external scheduler status were not independently verified here.